



OBJECT ORIENTED ANALYSIS AND DESIGN

Ch - 11

Analysis Packages

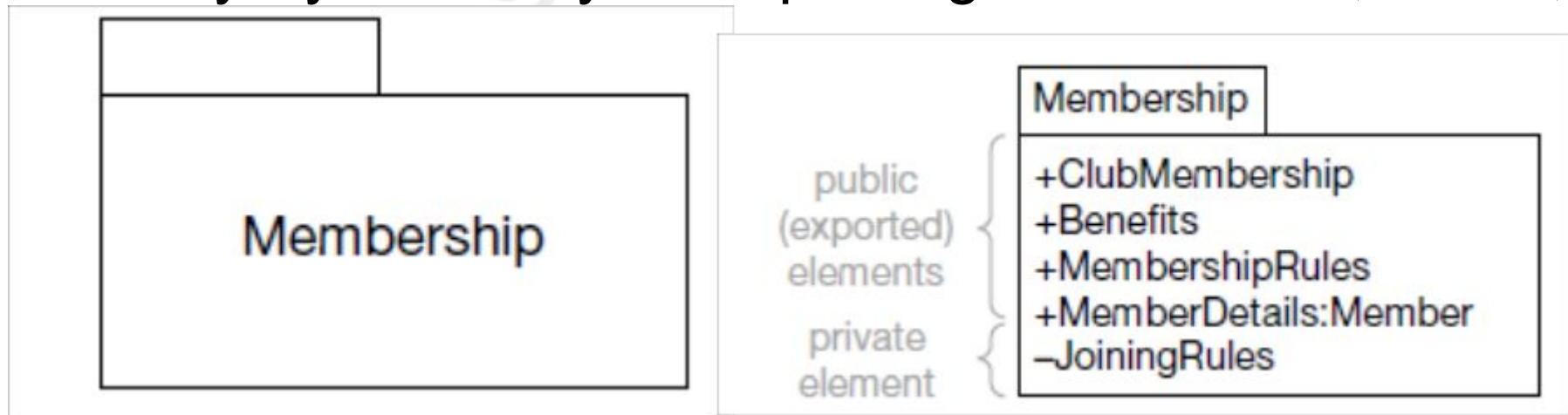
A study material for the students of GLS University

What is a package?

A Package is a folder in UML used to group related elements together.

Purpose of a Package:

- To group related elements (like classes, diagrams, or components).
- To create a semantic boundary (clear separation in model).
- Helps in teamwork – allows multiple people to work in parallel.
- Keeps names unique inside one package.
- Example:
- A Library System may have packages like Books, Users,



Package Syntax

A package looks like a folder icon.

The name of the package can appear on the tab or inside the folder.

You can give visibility to elements inside a package:

Symbol	Visibility	Semantics
+	public	Elements with public visibility are visible to other packages
-	private	Elements with private visibility are completely hidden inside the package
#	protected	Elements with protected visibility are only visible to packages that are children of their package

Packages and Namespaces

A **Namespace** is a space that defines a unique naming boundary.

Inside it, all element names must be unique.

Key Points:

- Each package acts as a namespace.
- If one package wants to use something from another package, it must use a qualified name (like a full path).

Example:

Library :: Users :: Librarian

→ means: the Librarian class is inside the Users package, which is inside Library.

Nested Packages

A Nested Package means one package inside another.

Features:

- Packages can be nested to any level.
- The inner (child) package can access elements from the outer (parent) package.
- To access items in another nested package, you use the fully qualified name.

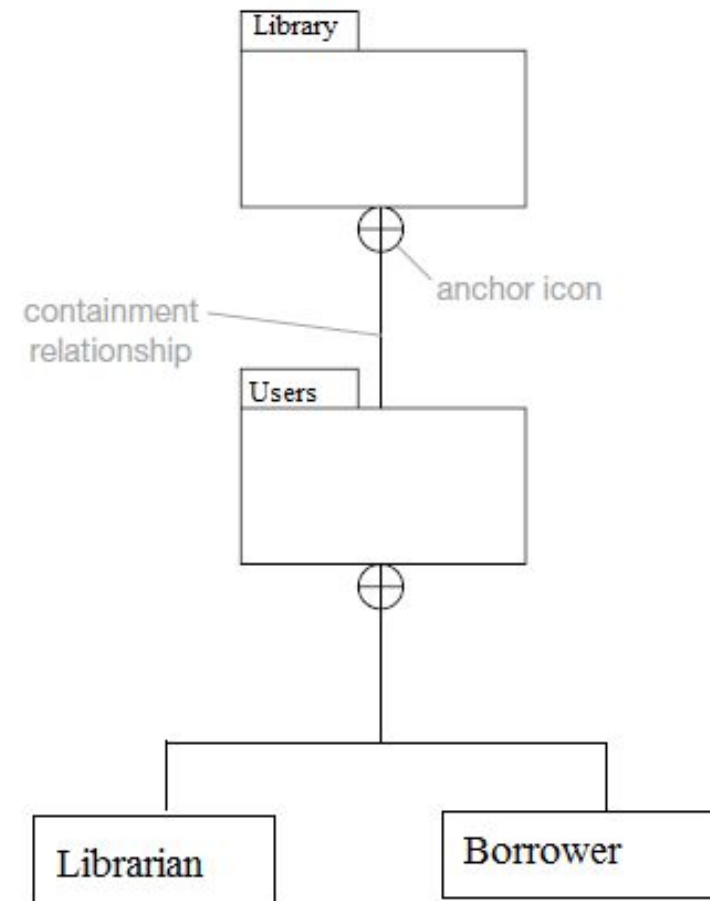
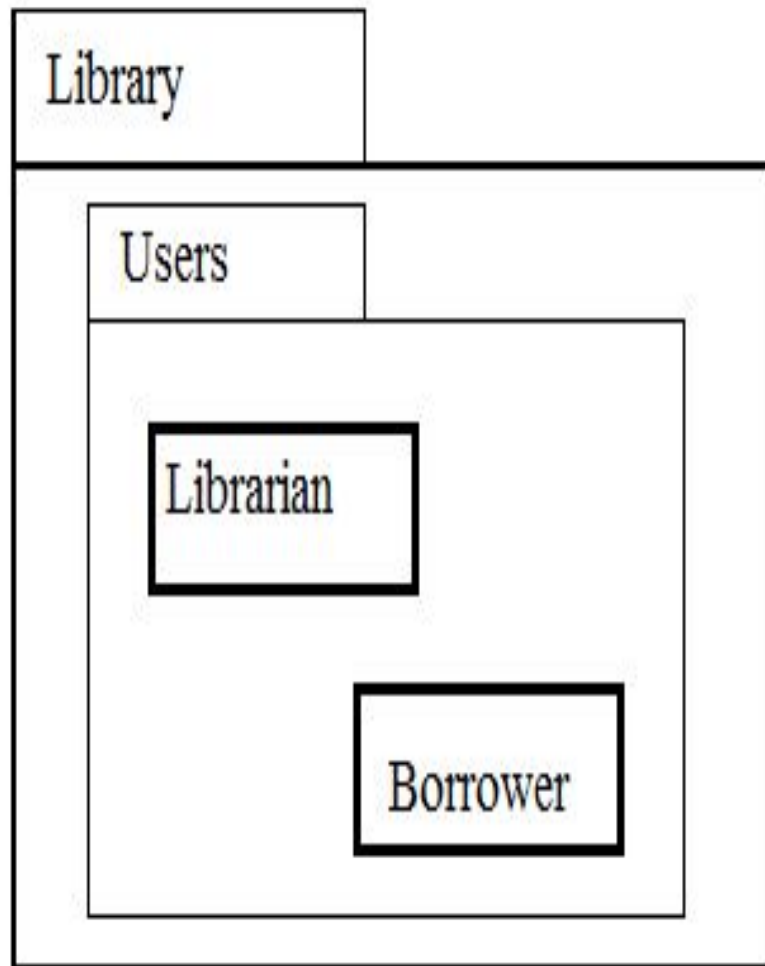
Example:

In package Library, you have a nested package Users with classes Librarian and Borrower.

So, to use them: Users::Librarian or Users::Borrower.

Nested Packages

An alternative nesting syntax:



Package Dependencies

A dependency means one package relies on another package for some elements.

There are four types of dependencies, each with a different meaning.

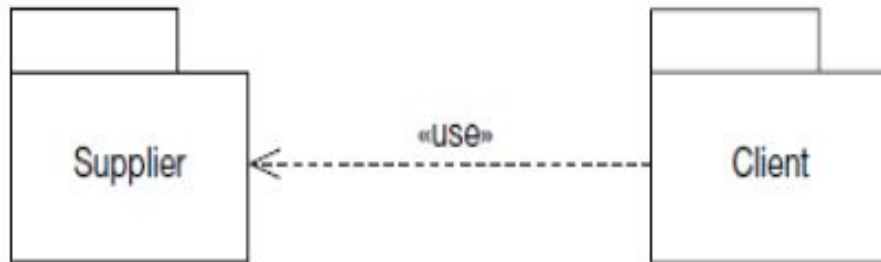
Common ones:

<<import>>

<<access>>

Package dependency

Semantics



An element in the client uses a public element in the supplier in some way – the client depends on the supplier

If a package dependency is shown without a stereotype, then «use» should be assumed



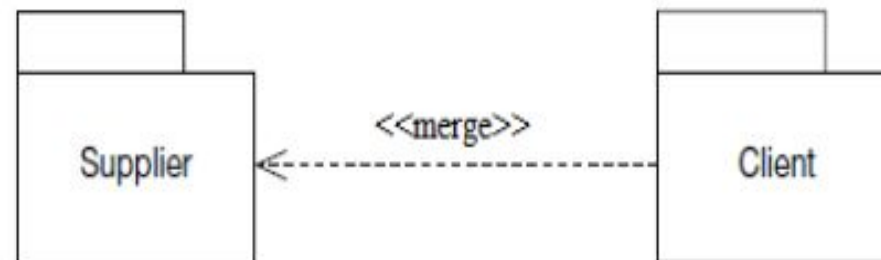
Public elements of the supplier namespace are added as public elements to the client namespace

Elements in the client can access all public elements in the supplier using unqualified names



Public elements of the supplier namespace are added as private elements to the client namespace

Elements in the client can access all public elements in the supplier using unqualified names



Public elements of the supplier package are merged with elements of the client package

you should not encounter this type in OO analysis and design

Import vs Access

Example:

If Package A imports Package B → A can use B's public classes.

If A only accesses B → A can use them privately, not share further.

Type	Meaning	Visibility
<<import>>	Public merge : all public elements of supplier become public in client	Public
<<access>>	Private merge : supplier's public elements become private in client	Private

Transitivity

If A depends on B and B depends on C $\rightarrow$ then A indirectly depends on C.

Key Points:

`<<import>>` is transitive

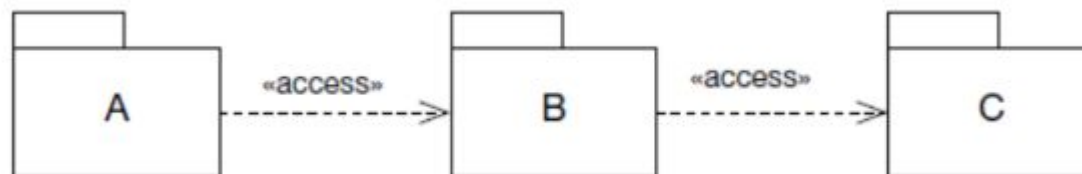
`<<access>>` is not transitive

Example:

If A imports B and B imports C $\rightarrow$ A can see C.

But if A accesses B and B accesses C $\rightarrow$ A cannot see C.

This helps control coupling (connections) between packages.



Managing Access

Public elements in Package C → become private in Package B

Public elements in Package B → become private in Package A

Hence, A can't see elements of C directly.

This rule helps to **control access** and keep the **system organized and secure**

Package Generalization

Package Generalization means one package (child) **inherits** from another package (parent).

- The **child package** gets all **public and protected** elements of its parent.
- It can **add new elements** or **override** existing ones.

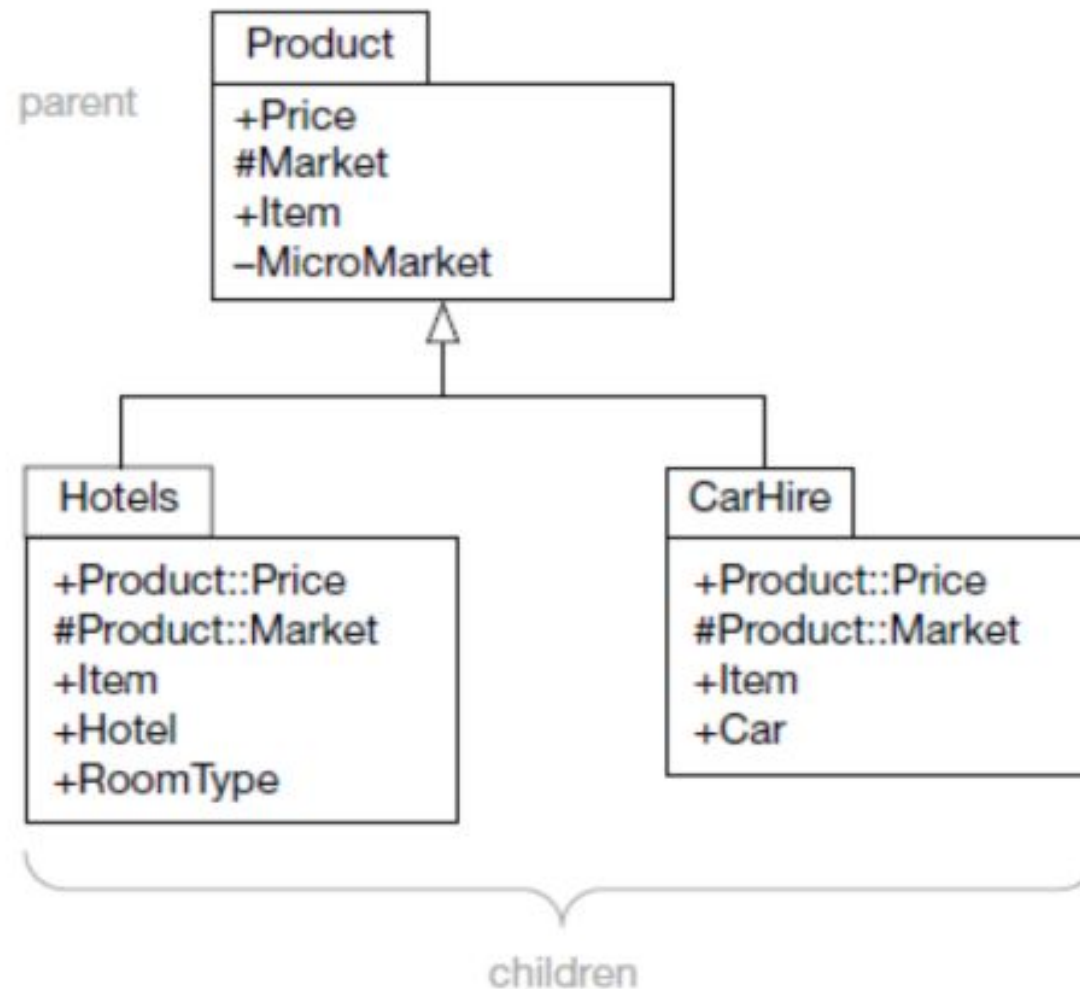
Example:

Parent Package → Product

Child Packages → Hotels, CarHire

- Both inherit items from Product.
- Hotels adds Hotel and RoomType classes.
- CarHire adds a Car class.
- Both override the Item class from the parent.

Package Generalization



GLS FCAIT MSC-(IT)

GLS FCAIT MSC-(IT)

GLS FCAIT MSc-(IT)

THANK YOU